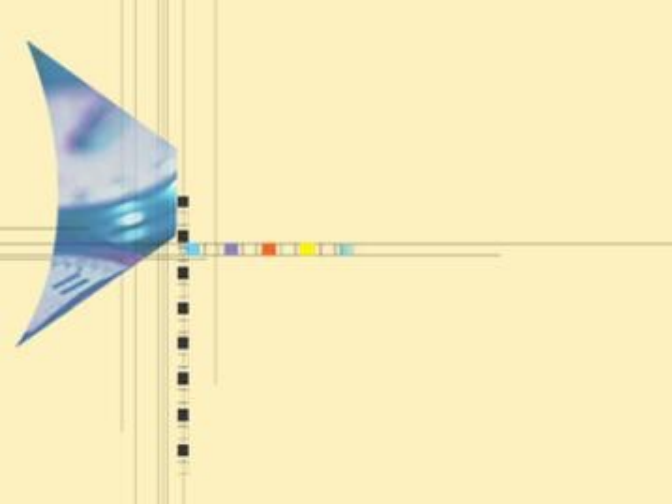




算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

周一(3-4，三区1-502)，周三（6-7，三区1-325）

2025/4/14

武汉大学计算机学院

武汉大学
Wuhan University





主要内容

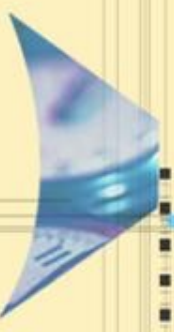
7.1 引言

7.2 判定问题

7.3 P类和NP类

7.4 多项式时间归约与NP完全性

7.5 NP完全性证明



主要内容

7.4 多项式时间归约与NP完全性

7.4.1 多项式时间归约

7.4.2 NP完全性

7.4.3 Cook-Levin 定理

7.4 多项式归约(Reduction)与NP完全性

- 对NP类的许多问题迄今既没有找到多项式时间的求解算法，也没能证明它们是难解的。既然如此，不妨换一种角度考虑，如果NP中存在难解问题，那么NP中最难的问题一定是难解的。那么什么是NP中最难的问题呢？
- 用术语“NP完全性” (*NP-complete*) 表示NP类的一个子类的判定问题，它们在下述意义上是最难的，即如果其中一个被证明能用多项式时间确定性算法求解，那么NP中的所有问题都可用多项式时间确定算法求解。也就是有 $NP = P$ 。
- 为了比较问题之间的难度，引入多项式归约的概念。

7.4.1 多项式时间归约

- 一个问题A可以归约为问题B的含义即是，可以用解决问题B的解法来解决问题A，或者说，问题A可以“变成”问题B。**直观意义**：B的时间复杂度高于或者等于A的时间复杂度。也就是说，问题A不比问题B难。
- 例如：一元一次方程可以“归约”为一元二次方程。意即知道如何解一个一元二次方程那么一定能解出一元一次方程。
- 可以写出两个程序分别对应两个问题，那么我们能找到一个“变换规则”，按照这个规则把解一元一次方程程序的输入数据变一下，用在解一元二次方程的程序上，两个程序总能得到一样的结果。这个规则即是：两个方程的对应项系数不变，一元二次方程的二次项系数为0。按照这个规则把前一个问题转换成后一个问题，两个问题就等价了。

7.4.1 多项式时间归约

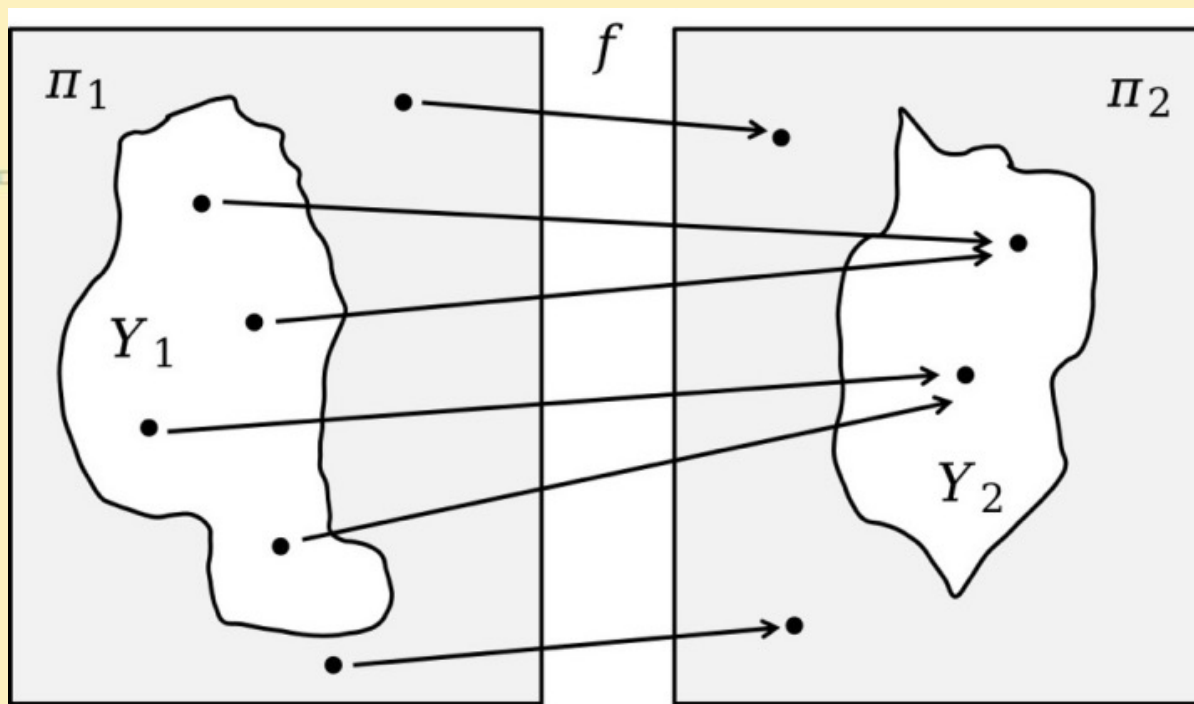
定义 设判定问题 $\Pi_1 = \langle D_1, Y_1 \rangle$ 和 $\Pi_2 = \langle D_2, Y_2 \rangle$, 如果存在一个函数 $f: D_1 \rightarrow D_2$ 满足如下条件:

(1) f 是多项式时间可计算的, 即存在计算 f 的多项式时间算法;

(2) 对所有 $I \in D_1$, $I \in Y_1 \Leftrightarrow f(I) \in Y_2$

则称 f 是 Π_1 到 Π_2 的多项式时间归约函数。计算 f 的多项式时间算法 F 称为归约算法。

定义 设判定问题 Π_1 和 Π_2 , 如果存在 Π_1 到 Π_2 的多项式时间归约算法, 则称 Π_1 可多项式时间归约到 Π_2 , 记作 $\Pi_1 \propto_{\text{poly}} \Pi_2$ 。



f 为一个多项式时间的映射，使得若 $I \in Y_1$ （回答 Yes），则 $f(I) \in Y_2$ （回答 Yes）。而且，若 $I \notin Y_1$ ，则 $f(I) \notin Y_2$ 。

因此，归约函数提供了从判定问题 Π_1 的任意实例 I 到判定问题 Π_2 的实例 $f(I)$ 上的映射。

如果能提供是否有 $f(I) \in Y_2$ 的答案，也就直接提供了是否有 $I \in Y_1$ 的答案。

多项式时间归约

例1: 哈密顿回路问题 (HC) $\propto_{\text{poly}}$ 货郎担问题 (TSP)

证明:

设 $\text{HC} = \langle D_1, Y_1 \rangle$, $\text{TSP} = \langle D_2, Y_2 \rangle$ 。定义函数 $f: D_1 \rightarrow D_2$, 对任意的 $I \in D_1$, I 是一个无向图 $G = \langle V, E \rangle$, $f(I)$ 为带权完全图 $G' = \langle V, E' \rangle$ 和一个界限值 $K = |V|$: V 代表城市集, E' 是 V 中所有城市之间边的集合, 即 $E' = \{(u, v) | u, v \in V\}$ 。 E' 中的每条边 (u, v) 赋予如下权值:

$$d(u, v) = \begin{cases} 1, & \text{若 } (u, v) \in E \\ 2, & \text{若 } (u, v) \notin E \end{cases}$$

显然, f 是多项式可计算的。

多项式时间归约

对任意 $f(I) \in D_2$ 中经过每个城市恰好一次的周游路线，总共有 $|V|$ 条边，每条边的长度为1或者2，因此周游路线的长度大于等于 K 。又由于如果 $f(I) \in Y_2$ ，则经过每个城市恰好一次的周游路线的长度必须小于等于 K 。因此，

$f(I) \in Y_2 \Leftrightarrow$ 经过每个城市恰好一次的周游路线的长度必须等于 K

$\Leftrightarrow$ 周游路线中每条边的权值为1

$\Leftrightarrow$ 周游路线中的每条边都是图 G 中的边

$\Leftrightarrow$ 周游路线是图 G 中的哈密顿回路

$\Leftrightarrow I \in Y_1$

即 $I \in Y_1 \Leftrightarrow f(I) \in Y_2$ 。因此 f 是多项式归约函数， $HC \propto_{\text{poly}} \text{TSP}$ 。

多项式时间归约的性质

多项式时间归约为比较问题之间的求解难度提供了强有力的工具，对判定问题 Π_1 和 Π_2 ，很容易证明如下结论（略）：

定理：如果 $\Pi_1 \propto_{\text{poly}} \Pi_2$ ，则 $\Pi_2 \in P$ 蕴含 $\Pi_1 \in P$ ； Π_1 是难解的蕴含 Π_2 是难解的。

也即表明：

定理：如果 $\Pi_1 \propto_{\text{poly}} \Pi_2$ ，则相对于多项式时间，如果 Π_2 是易解的，则 Π_1 也是易解的，即 Π_2 不会比 Π_1 更容易。或者反过来讲，即 Π_1 不会比 Π_2 更难。

另外，也容易证明多项式归约具有传递性：

定理：设 Π_1 、 Π_2 和 Π_3 是三个判定问题，且有 $\Pi_1 \propto_{\text{poly}} \Pi_2$ ， $\Pi_2 \propto_{\text{poly}} \Pi_3$ ，则 $\Pi_1 \propto_{\text{poly}} \Pi_3$ 。

7.4.2 NP完全性

定义 对于一个判定问题 Π ，如果对所有 $\Pi' \in NP$ ，都有 $\Pi' \propto_{\text{poly}} \Pi$ ，则称 Π 是NP难的（NP-hard, NPH）。

含义：NPH问题不会比NP类中的任何问题容易。

定义 对于一个判定问题 $\Pi \in NP$ ，如果对所有 $\Pi' \in NP$ ，都有 $\Pi' \propto_{\text{poly}} \Pi$ ，则称 Π 是NP完全的（NP-complete, NPC）。

含义：NPC问题是NP类中的NPH问题。由此也可以推断，NPC问题是NP类中最难的问题。NP完全性是判定P是否等于NP的关键。

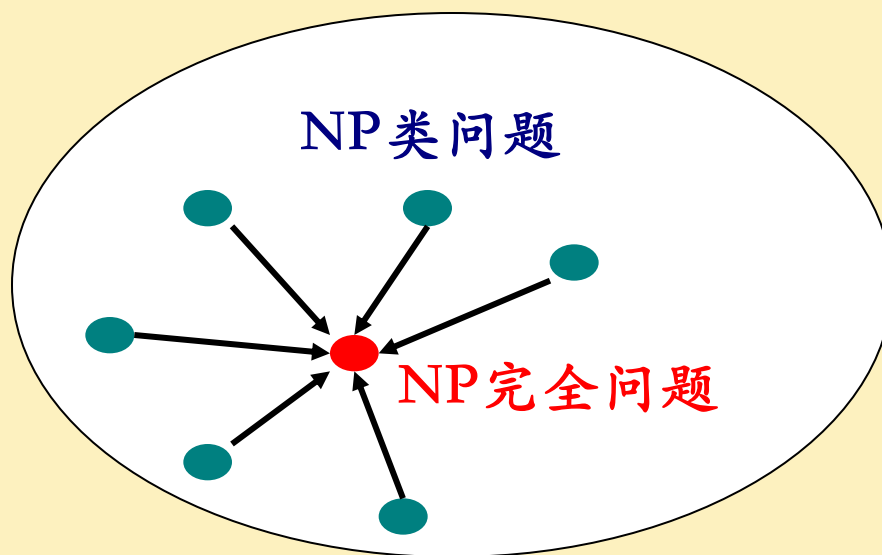
NPH问题和NPC问题的唯一差别是：NPC问题必须是属于NP类的问题，而NPH问题不一定在NP类中。

NP完全类

NP完全问题是一类具备如下特殊性质的NP类问题

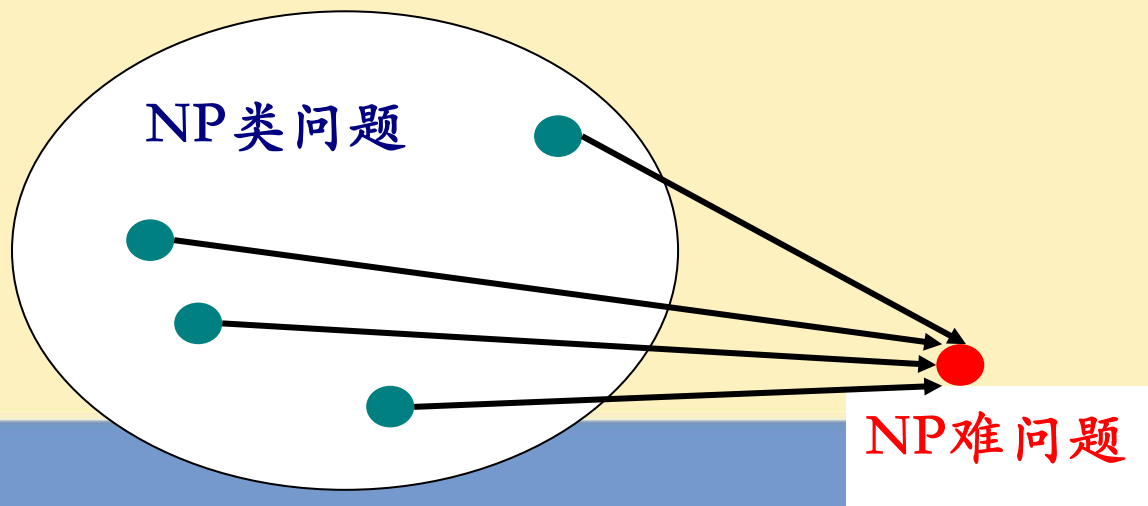
(1) Π (该问题本身) 就是一个NP类问题

(2) 每一个NP类问题都可以通过多项式时间归约到 Π



NP困难问题

还有一类问题，它们中的任何一个 Π ，虽然也能证明所有的NP类问题可以在多项式时间内变换到问题 Π ，**但不能证明 Π 也是NP类问题**，所以 Π 不是NP完全的。但问题 Π 至少与任意NP类问题有同样的难度，这样的问题称为NP困难问题。

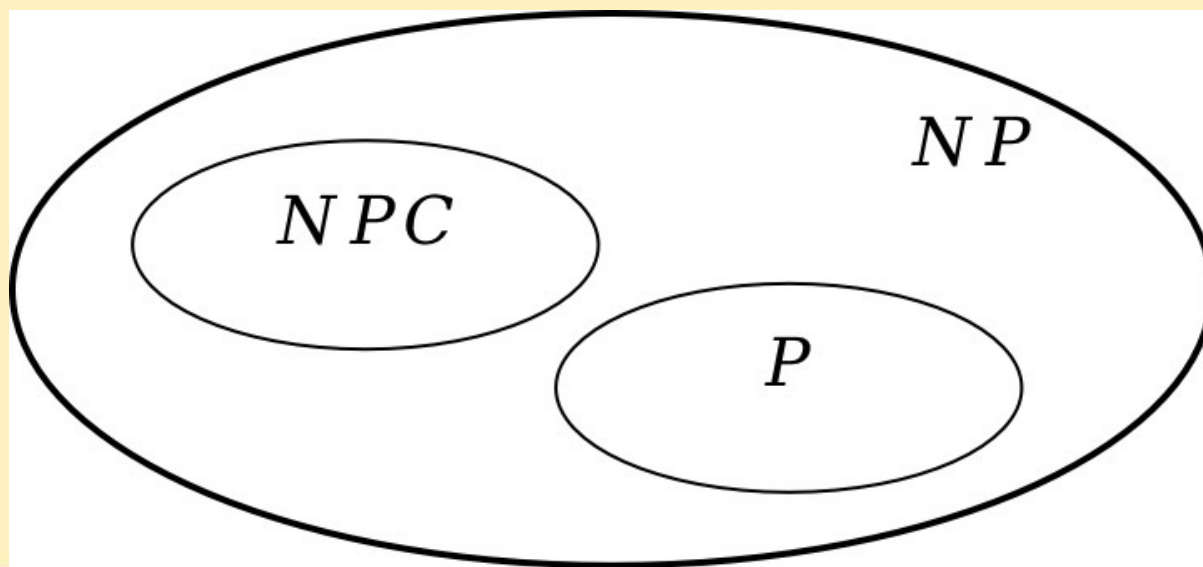


对“NP完全问题”的评述

- NP完全问题是NP类问题中最难的一类问题，至今已经发现了几千个，但没有一个找到多项式时间算法。
- 如果某一个NP完全问题能在多项式时间内解决，则每一个NP完全问题都能在多项式时间内解决。
- 这些问题也许存在多项式时间算法，因为计算机科学是相对新生的科学，肯定还有新的算法设计技术有待发现；
- 这些问题也许不存在多项式时间算法，但目前缺乏足够的依据来证明这一点。

P、NP、NPC关系

- 普遍认为 P 、 NP 、 NPC 之间的关系如下图所示。
- 但是如果未来真能找到一个 NPC 问题的多项式时间算法，就能证明 $P = NP$ 。然而迄今并没有找到任何 NPC 问题的多项式时间算法。
- 因此在目前，证明一个问题是 NPC 问题就可认为该问题是难解的问题。



7.4.3 Cook-Levin 定理/库克-列文定理

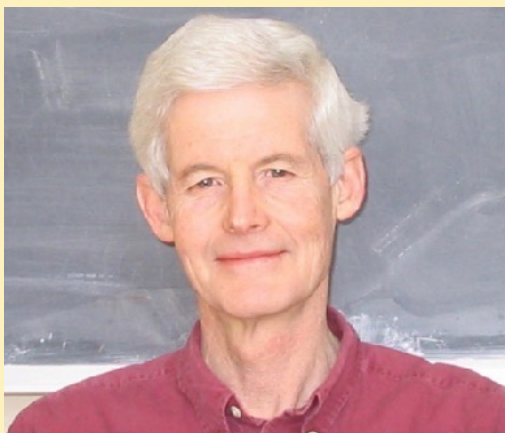
由于多项式时间可归约关系具有传递性，再结合NPH和NPC的定义，很容易证明如下两个定理：

定理（NP困难的传递性） 如果存在 $\Pi' \in \text{NPH}$ ，使 $\Pi' \propto_{\text{poly}} \Pi$ ，则 Π 是NPH的。

定理（NP完全性的传递性） 如果 $\Pi \in \text{NP}$ ，且存在 $\Pi' \in \text{NPC}$ ，使得 $\Pi' \propto_{\text{poly}} \Pi$ ，则 Π 是NPC的。

- 上述定理表明，如果要证明一个问题 Π 是属于NPH（NPC）的，不再需要证明所有NP问题都多项式时间可归约到 Π ，而是只需要证明存在一个NPH（NPC）的问题 Π' ， Π' 多项式时间可归约到 Π 即可。
- 因此，要证明一个问题 Π 是属于NPC的，现在只需要证明两点：
(1) $\Pi \in \text{NP}$ ； (2) 存在一个 $\Pi' \in \text{NPC}$ ，使得 $\Pi' \propto_{\text{poly}} \Pi$ 。
- 哪些问题是NPC问题？ 是否存在NPC问题？

可满足性问题-第一个被证明的NPC问题



Stephen A. Cook



Leonid A. Levin

- 美国理论计算机科学家斯蒂芬·库克 (Stephen A. Cook) 和前苏联理论计算机科学家莱昂尼德·列文 (Leonard A. Levin) 各自独立发现了第一个NPC问题, 即布尔可满足性问题, 证明了NPC问题存在。

可满足性问题-第一个被证明的NPC问题

- 可满足性问题即合取范式的可满足性问题，来源于许多实际的逻辑推理的应用。合取范式形如 $A_1 \wedge A_2 \wedge \dots \wedge A_n$ ，其中子句 A_i ($1 \leq i \leq n$)形如： $a_1 \vee a_2 \vee \dots \vee a_k$ ，其中 a_j 称为文字，为布尔变量或它的否定。一个子句是文字的析取。
- SAT问题是指：是否存在一组对所有布尔变量的赋值，使得整个合取范式为真？
- 例如： $f = (x_1 \vee x_2) \wedge (\bar{x}_1 \vee x_3 \vee x_4 \vee \bar{x}_5) \wedge (x_1 \vee \bar{x}_3 \vee x_4)$

当 x_1 和 x_3 都为真、其余文字任意赋值时， f 值为真，因此 f 是可满足的。

Cook-Levin 定理/库克-列文定理

- 判定问题: SAT.
输入: 一个合取范式的布尔公式 f .
问题: f 是可满足的吗?
- **定理** (Cook-Levin 定理): SAT 问题是 NP 完全的
- 既然 SAT 已被证明是 NPC 问题, 那么任何可以多项式时间归约到 SAT 的 NP 问题也是 NPC 问题。
- 已经被证明的新的 NPC 问题又可以用来证明更多其他的 NPC 问题。。。

NP完全问题

- 第一个NP完全问题 (Cook定理 1971)
 - 可满足性问题是NP完全问题
- 1972年, Karp证明了十几个问题都是NP完全的。
 - 3SAT, 3DM, VC, 团, HC, 划分
- 更多的NP完全问题
 - 1979年: 300多个
 - 1998年: 2000多个
- 这些NP完全问题的证明思想和技巧, 以及利用他们证明的几千个NP完全问题, 极大地丰富了NP完全理论。



主要内容

7.1 引言

7.2 判定问题

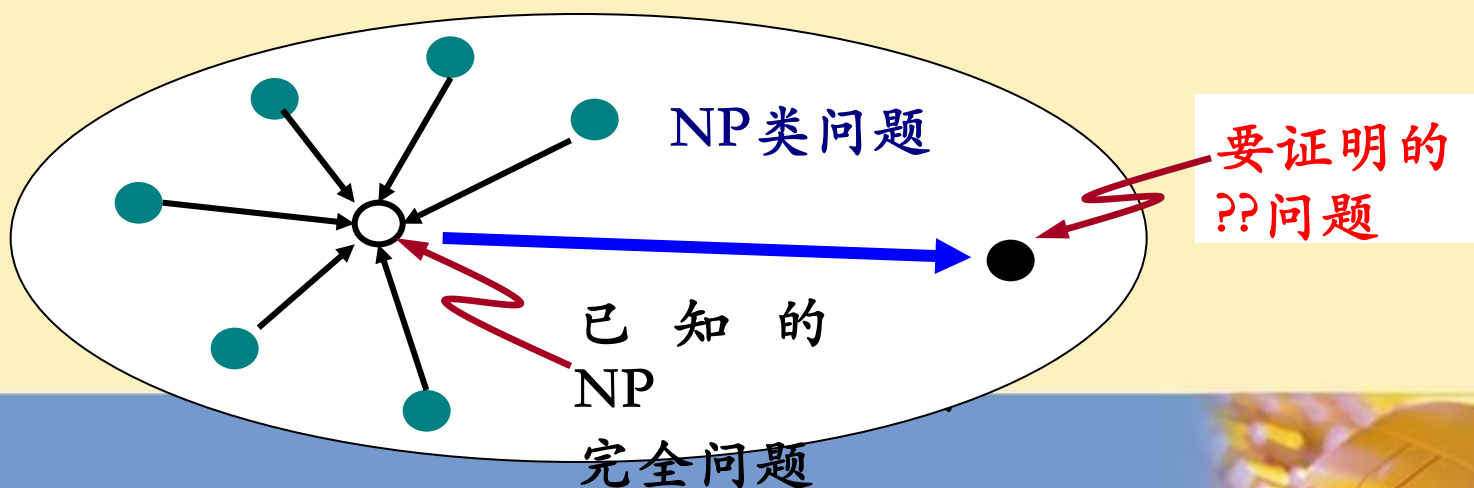
7.3 P类和NP类

7.4 多项式时间归约与NP完全性

7.5 NP完全性证明

如何证明一个问题是NP完全的?

- 根据NP完全问题的定义（满足的两个性质），显然地，证明需要分两个步骤进行：
 - 证明问题II是NP类问题；即可以在多项式时间内以确定性算法验证一个任意生成的串，以确定它是否为问题的一个解。
 - 证明NP类问题中的每一个问题都能在多项式时间内变换为问题II。由于多项式问题变换具有传递性，所以，只需证明一个已知的NP完全问题能够在多项式时间内变换为问题II。



如何证明一个问题是NP完全的？

- 既然现在已经知道了很多NP完全类的问题，那么，根据NP完全性的传递性可知，为了证明一个新的问题 Π 是NP完全的，仅需要证明以下两个条件同时成立：

(1) $\Pi \in \text{NP}$;

(2) 存在着一个NP完全问题 Π' ，使 $\Pi' \propto_{\text{poly}} \Pi$ 。

举例

例2: 已知HC问题是NPC问题, 证明TSP问题也是NPC问题。

证明:

(1) $TSP \in NP$

可设计如下多项式不确定性运行算法: 对任意给定的一个边带权的完全无向图 G 和正整数 K , 随机猜测 G 中所有顶点的一个排列代表一条恰好经过每个城市一次的周游路线, 然后检查这个排列相邻的两个顶点之间边以及排列的首尾顶点之间边的总权值之和是否不超过 K 。如果是, 则回答“Yes”, 否则回答“No”。当 G 中存在长度不超过 K 的周游路线时, 总能猜对一个顶点的排列, 它确实给出一条长度不超过 K 的周游路线, 从而使算法回答“Yes”; 如果 G 中不存在长度不超过 K 的周游路线, 则猜测的任何排列表示的周游路线长度都会大于等于 K , 从而算法总是回答“No”。显然, 猜测一个排列和检查这个排列对应的周游路线的长度是否不超过 K 都可以在多项式时间内完成。因此, TSP是NP类问题。任何一个构成长度不超过 K 的周游路线的顶点排列都是证据。

(2) $HC \propto_{poly} TSP$, 例1已证。



关于多项式时间归约的常用证明方法

- 限制法
- 局部替换法
- 构件设计法
- 问题等价性



7.5.1 限制法

- 限制法证明是最简单、几乎最常用的一种方法。如果限制问题 Π 的实例满足某些条件时得到的子问题 Π' 是已知的NP完全问题，那么就很容易将 Π' 多项式时间归约到 Π ，这就是限制法。
- 换句话说，用限制法证明给定问题 Π ($\Pi \in \text{NP}$) 是否属于NPC问题，就是证明 Π 包含一个已知为NP完全的问题 Π' 作为它的一种特殊情况 (Π' 是 Π 的子问题或者可以看作是 Π 的子问题)。
- 当然， Π' 在形式上不一定就是原问题 Π 的子问题，而是等价原问题的某个子问题，因此可以看作是原问题的子问题。

例：证明最大可满足性问题是NP完全问题

- 最大可满足性 (Max-SAT)问题：任给关于 n 个变元 $x_1, x_2, \dots, x_n$ 的 m 个简单析取式 $C_1, C_2, \dots, C_m$ 以及正整数 K ，问是否存在关于这些变元的赋值 t 使得 $C_1, C_2, \dots, C_m$ 中至少有 K 个为真？
- 对比Max-SAT问题和SAT问题，很容易看出SAT是Max-SAT问题限定 $K = m$ 的特殊情况。
 - 当 $K = m$ 时，赋值 t 使 $C_1, C_2, \dots, C_m$ 中至少有 K 个为真就是使 $C_1, C_2, \dots, C_m$ 全部为真，这当且仅当赋值 t 使 $F = C_1 \wedge C_2 \wedge \dots \wedge C_m$ 为真。
 - 因此当 $K = m$ 时，Max-SAT问题就退化为SAT问题。即SAT可以看作是Max-SAT的子问题。

$\text{SAT} \propto_{\text{poly}} \text{Max-SAT}$

- 由于SAT可以看作是Max-SAT的子问题，因此容易将SAT多项式时间规约到Max-SAT。
- 对每一个 $I \in \text{SAT}$ ， I 是关于变元 $x_1, x_2, \dots, x_n$ 的合取范式 $F = C_1 \wedge C_2 \wedge \dots \wedge C_m$ ，其中 $C_1, C_2, \dots, C_m$ 是简单析取式。设计转换函数 f ， $f(I)$ 由 m 个简单析取式 $C_1, C_2, \dots, C_m$ 以及正整数 $K = m$ 构成。显然 f 是多项式可计算的。
- 赋值 t 使 F 为真当且仅当 t 使得 $C_1, C_2, \dots, C_m$ 全部为真，从而对 I 回答“**Yes**”当且仅当对 $f(I)$ 回答“**Yes**”。
- 因此 $\text{SAT} \propto_{\text{poly}} \text{Max-SAT}$ 。

7.5.2 局部替换法

- 如果子问题是NP完全问题，包含子问题的原问题的NP完全性用限制法很容易来证明。反过来，如果已知一个问题是NP完全的，要证明它的子问题（子问题是NP的）也是NP完全的就没有那么容易，这时可以用局部替换法来证明。
- 一般来讲，如果已知 $\Pi \in \text{NPC}$ ， Π' 是 Π 在限定一定条件下的子问题，或者更一般情况， Π' 与 Π 的问题实例的构造有某方面的相似之处时，往往可以采用局部替换法来证明 Π' 的NP完全性。
- 局部替换法选择已知NP完全问题的实例中的某些元素作为基本单元，然后把每个基本单元替换成指定结构，从而得到目标问题的对应实例。

例：证明三元可满足性问题是NP完全问题

- 三元可满足性 (3SAT) 问题：任给一个三元合取范式 F （每一个简单析取式恰好有三个文字组成），问 F 是可满足的吗？
- 显然，当限制SAT实例中的合取范式为三元合取范式时，得到3SAT问题。3SAT问题是SAT问题的子问题。



$\text{SAT} \propto_{\text{poly}} 3\text{SAT}$

- 对每一个 $I \in \text{SAT}$, I 是关于变元 $x_1, x_2, \dots, x_n$ 的合取范式 $F = C_1 \wedge C_2 \wedge \dots \wedge C_m$, 其中 $C_1, C_2, \dots, C_m$ 是简单析取式。
- 设计转换函数 f , 使得 $f(F) = F' = F'_1 \wedge F'_2 \wedge \dots \wedge F'_m$, 其中 F'_i 是对应于 C_i 的三元合取范式, 且 C_i 是可满足的当且仅当 F'_i 是可满足的。这样构造的 F' 显然满足 F 是可满足的当且仅当 F' 是可满足的。
- 根据 C_i 的四种可能情形, 三元合取范式 F'_i 分别构造如下:

$\text{SAT} \propto_{\text{poly}} \text{3SAT}$

(1) C_i 仅包含单个文字, 设 $C_i = z_1$, 其中 z_1 为文字, 即某变元或者变元的否定 (下同)。

引入两个新的变元 y_{i1} 和 y_{i2} , 令

$$F'_i = (z_1 \vee y_{i1} \vee y_{i2}) \wedge (z_1 \vee \neg y_{i1} \vee y_{i2}) \wedge (z_1 \vee y_{i1} \vee \neg y_{i2}) \wedge (z_1 \vee \neg y_{i1} \vee \neg y_{i2})$$

显然, 无论怎么赋值, $y_{i1} \vee y_{i2}$ 、 $\neg y_{i1} \vee y_{i2}$ 、 $y_{i1} \vee \neg y_{i2}$ 和 $\neg y_{i1} \vee \neg y_{i2}$ 不可能同时为真, 因此 F'_i 为真当且仅当 z_1 为真, 也即 F'_i 为真当且仅当 C_i 为真。

$\text{SAT} \propto_{\text{poly}} 3\text{SAT}$

(2) C_i 是两个文字的析取式, 设 $C_i = z_1 \vee z_2$, 其中 z_1 、 z_2 均为文字。

引入一个新的变元 y_i , 令

$$F'_i = (z_1 \vee z_2 \vee y_i) \wedge (z_1 \vee z_2 \vee \neg y_i)$$

显然, 无论怎么赋值, y_i 和 $\neg y_i$ 不可能同时为真, 因此 F'_i 为真当且仅当 $z_1 \vee z_2$ 为真, 也即 F'_i 为真当且仅当 C_i 为真。

(3) C_i 是三个文字的析取式, 设 $C_i = z_1 \vee z_2 \vee z_3$, 其中 z_1 、 z_2 、 z_3 均为文字。

直接令 $F'_i = C_i = z_1 \vee z_2 \vee z_3$, 显然 F'_i 为真当且仅当 C_i 为真。

SAT $\propto_{\text{poly}}$ 3SAT

(4) C_i 是三个以上文字的析取式, 设 $C_i = z_1 \vee z_2 \vee \cdots \vee z_k$, $k \geq 4$, 其中 z_1 、 z_2 、 $\cdots z_k$ 均为文字。

引入 $k-3$ 个新的变元 $y_{i1}, y_{i2}, \cdots, y_{i(k-3)}$, 令

$$F'_i = (z_1 \vee z_2 \vee y_{i1}) \wedge (\neg y_{i1} \vee z_3 \vee y_{i2}) \wedge (\neg y_{i2} \vee z_4 \vee y_{i3}) \wedge \cdots \wedge (\neg y_{i(k-4)} \vee z_{k-2} \vee y_{i(k-3)}) \wedge (\neg y_{i(k-3)} \vee z_{k-1} \vee z_k)$$

设由真值指派 t 满足 C_i , 则一定存在一个 $j (1 \leq j \leq k)$ 使得 $t(z_j) = 1$ 。将 t 扩展到对所有新变元的赋值上:

(4-1) 如果 $j = 1$ 或者 2 时, 令 $t(y_{is}) = 0$, $1 \leq s \leq k-3$;

(4-2) 如果 $j = k-1$ 或者 k 时, 令 $t(y_{is}) = 1$, $1 \leq s \leq k-3$;

(4-3) 如果 $3 \leq j \leq k-2$ 时, 令

$$t(y_{is}) = \begin{cases} 1, & 1 \leq s \leq j-2 \\ 0, & j-1 \leq s \leq k-3 \end{cases}$$

不难验证, 扩展后的真值指派 t 也满足 F'_i 。因此, 如果 $t(C_i) = 1$, 则 $t(F'_i) = 1$

$\text{SAT} \propto_{\text{poly}} 3\text{SAT}$

反过来，设 $t(F'_i) = 1$ 。如果 $t(y_{i1}) = 0$ ，则必有 $t(z_1 \vee z_2) = 1$ ；如果 $t(y_{i(k-3)}) = 1$ ，则必有 $t(\vee z_{k-1} \vee z_k) = 1$ ；否则必有 $s(1 \leq s \leq k-4)$ ，使得 $t(y_{is}) = 1$ 且 $t(y_{i(s+1)}) = 0$ ，从而 $t(z_{s+2}) = 1$ （反证可得）。因此，如果 $t(F'_i) = 1$ ，则必存在 $j(1 \leq j \leq k)$ 使得 $t(z_j) = 1$ ，从而如果 $t(C_i) = 1$ 。

因此， F'_i 为真当且仅当 C_i 为真成立。

综合四种情况可得， F 是可满足的当且仅当 F' 是可满足的。另外， F'_i 为三元合取范式，所包含的简单析取式的个数不会超过 C_i 中文字的4倍，每个析取式由3个文字组成。因此， $f(F) = F'$ 可在 $|F|$ 的多项式时间内构造，也就函数 f 是多项式时间可计算的。因此 f 是 SAT 到 3SAT 的多项式归约函数。

$\text{SAT} \propto_{\text{poly}} 3\text{SAT}$ 得证。

- 上述在使用局部替换法进行变换中，SAT实例中要替换的基本单元是子句，每一个子句（基本单元）都按照同样规则替换成一个子句集（构成合取式），每一个替换仅构成结构的局部修改。除去反映原来实例中未改变的部分，这些替换在本质上是相互独立的。
- 3SAT具有简单整齐的结构，因此在NP完全性证明中经常被选作已知的NP完全问题。

7.5.3 构件设计法

- 当已知为NP完全问题的是要证明的目标问题（NP问题）的子问题，或者形式上不一定就是子问题，但可以看作是目标问题的子问题时，通常采用限制法证明；
- 当要证明的目标问题（NP问题）是已知的NP完全问题的子问题，或者两类问题的实例的构造有某方面的相似之处时，通常采用局部替换法证明。
- 如果两种方法都不行，则需要考虑构件法证明。构件设计法是相对较复杂的一种证明方法，往往需要很强的技巧。它的基本思想是用目标问题的实例设计某些“构件”，能够把这些“构件”组合在一起“实现”已知的NP完全问题的实例。

顶点覆盖、独立集和团集问题

- 顶点覆盖:
 - 给出一个无向图 $G=(V,E)$ 和一个正整数 k , 是否存在大小为 k 的子集 $C \subseteq V$, 使 E 中的每条边至少和 C 中的一个顶点关联?
- 独立集:
 - 独立集: 给出一个无向图 $G=(V,E)$ 和一个正整数 k , 是否存在 k 个顶点的子集 $S \subseteq V$, 使得对于每一对顶点 $u, w \in S$, $(u,w) \notin E$?
- 团集:
 - 团集: 给出一个无向图 $G=(V,E)$ 和一个正整数 k , G 包含一个大小为 k 的团集吗?(注意一个 G 中大小为 k 的团集是 G 中 k 个顶点的一个完全子图。)
- 容易证明所有这三个问题确实是NP的。

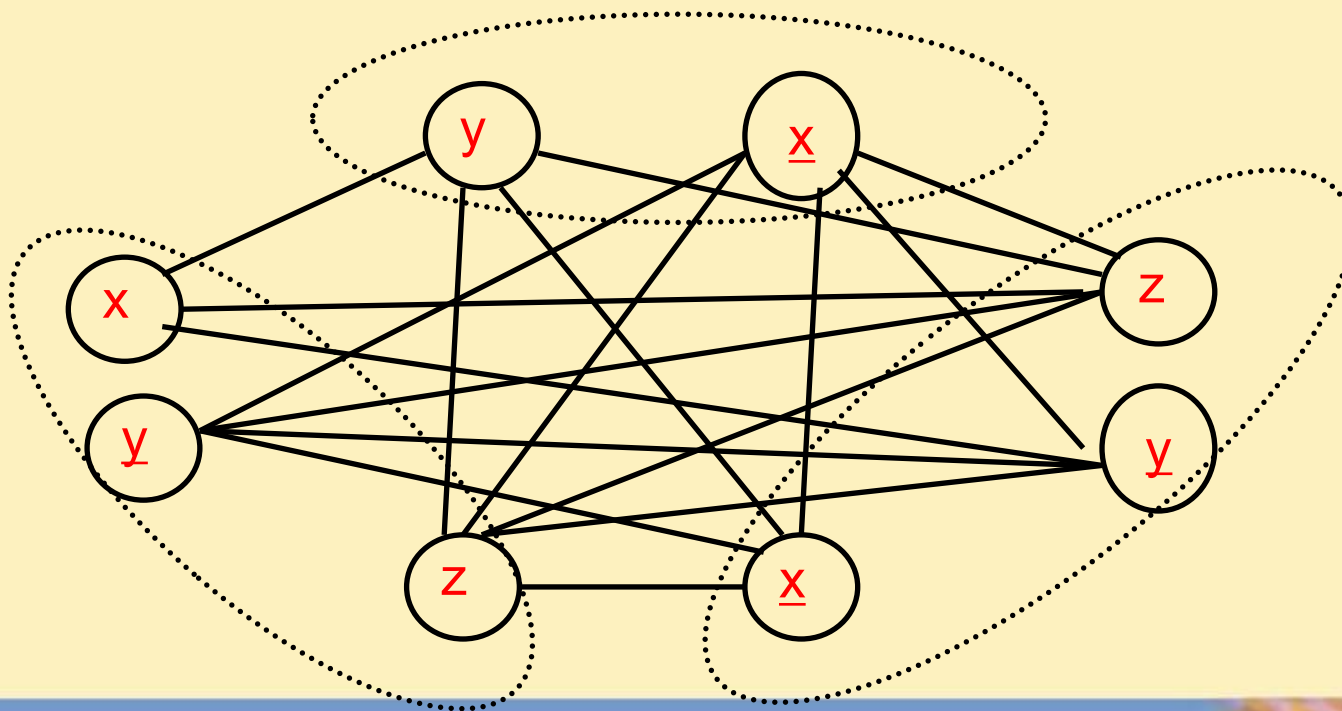
可满足性 $\in_{\text{poly}}$ 团集问题

- 给出一个有 m 个子句和 n 个布尔变元 $x_1, x_2, \dots, x_n$ 的可满足性实例 $f = C_1 \wedge C_2 \wedge \dots \wedge C_m$ ，构造一个图 $G = (V, E)$ ，其中 V 是 $2n$ 个文字的所有出现的集合(注意一个文字是一个布尔变元或它的否定)，并且 $E = \{(x_i, x_j) \mid x_i \text{ 和 } x_j \text{ 位于两个不同的子句, 并且 } x_i \neq \sim x_j\}$
- 图 G 的每个顶点对应于 f 中的每个文字，多次出现的重复表示
- 若图 G 中两个顶点对应的文字不互补，且不出现在同一子句中，则将其连线。（节点 a - b 连线意味着文字 a 和 b 可同时为真）

显然，上述构造图 G 的方法可在多项式时间内完成

可满足性 ∞_{poly} 团集问题

$$f = (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee y) \wedge (\bar{x} \vee \bar{y} \vee z)$$



可满足性 α_{poly} 团集问题

引理: f 是可满足的当且仅当 G 有一个大小为 m 的团集。

证明:

设 f 有 m 个子句, 则有:

f 可满足 $\Leftrightarrow m$ 个子句同时为真

$\Leftrightarrow$ 每个子句至少1个文字为真

$\Leftrightarrow G$ 中有 m 个顶点之间彼此相连

$\Leftrightarrow G$ 中有 m 个顶点的团集



上述构件设计法比较简单：

- 根据简单析取式设计构件，这些构件即图中的顶点。
- 构件以及构件之间的连接组成了图 G 。
- 通过这种方式达到用团集实例表达SAT实例的目的。



可满足性 $\propto_{\text{poly}}$ 顶点覆盖

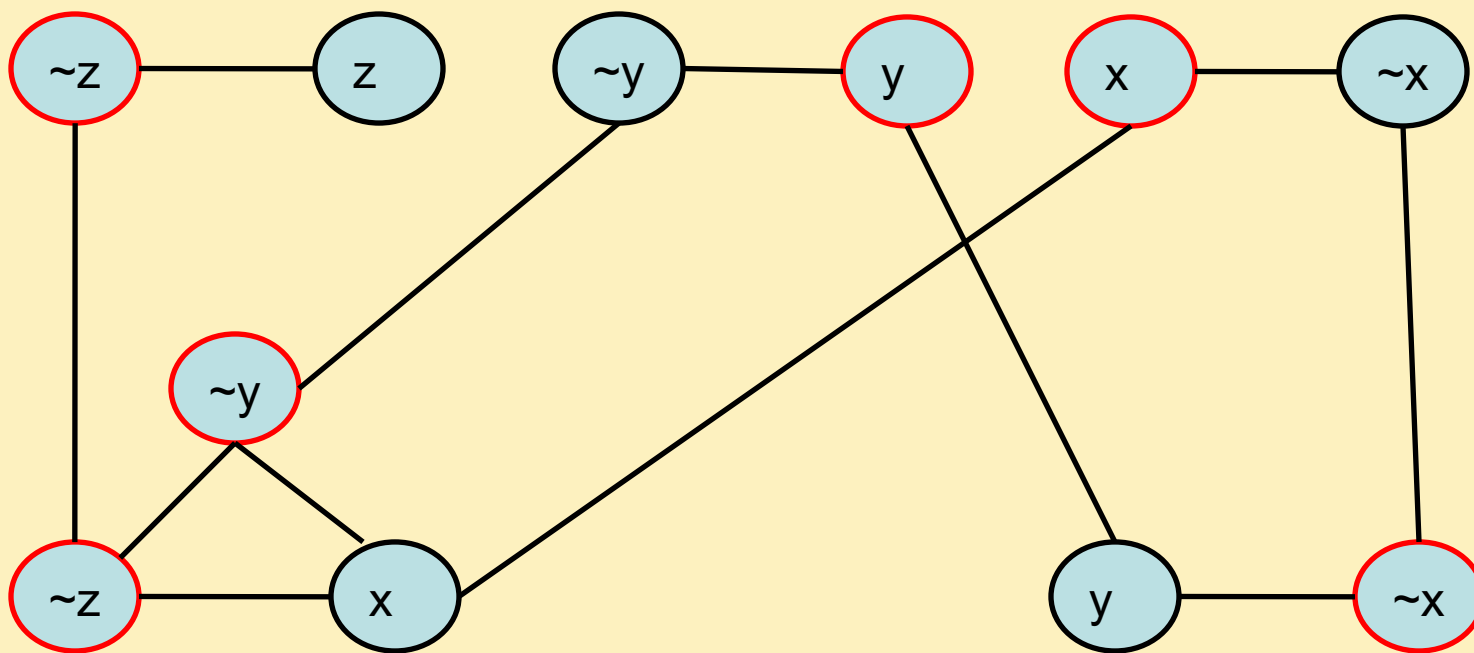
给出一个有 m 个子句和 n 个布尔变元 $x_1, x_2, \dots, x_n$ 的可满足性实例 $f = C_1 \wedge C_2 \wedge \dots \wedge C_m$ ，按如下方式构造图 G ：

1. 对于 f 中的每一个布尔变元 x_i ， G 包含一对顶点 x_i 和 $\sim x_i$ ，它们有一条边相连；
2. 对于每个子句 C_j 包含的 n_j 个文字， G 包含一个大小为 n_j 的团集 C_j ；
3. 对于在 C_j 中的每个顶点 w ，有一条边连接 w 到(1)中构造的顶点对 $(x_i, \sim x_i)$ 中它相应的文字。这些边称为连通边；
4. 令
$$k = n + \sum_{j=1}^m (n_j - 1)$$

显然，上述构造图 G 的方法可在多项式时间内完成

可满足性 ∞_{poly} 顶点覆盖

$$f = (x \vee \bar{y} \vee \bar{z}) \wedge (\bar{x} \vee y)$$



可满足性 ∞_{poly} 顶点覆盖

引理： f 是可满足的当且仅当构造的图 G 有一个大小为 k 的顶点覆盖集。

证明： $\Rightarrow$

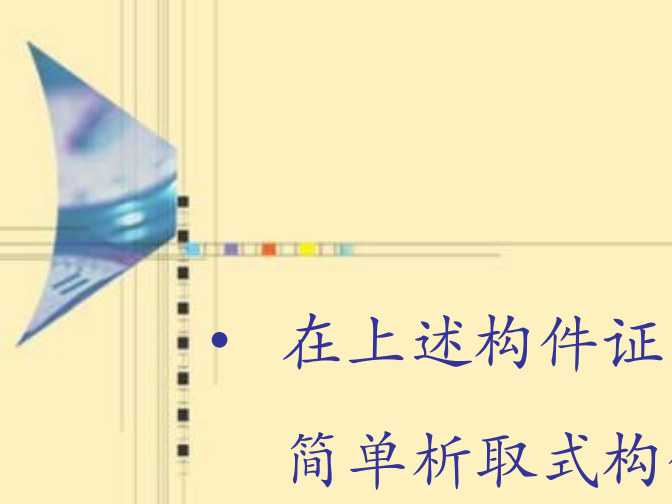
- 考察 f 中每个文字的赋值：如果 x_i 为真，则将步骤 (1) 构造的对应顶点 x_i 加到顶点覆盖集中；否则，将 $\sim x_i$ 对应顶点加入到顶点覆盖集中；因此在 (1) 中所有边也已经被覆盖。
- 由于 f 是可满足的，因此每个子句中至少有 1 个文字赋值为真。在步骤 (2) 构造的每个子句对应的团集 C_j 中与赋值为真的文字对应的顶点 u ，同 (1) 中对应的顶点 v 之间的连通边 (u, v) 均已经被覆盖（因为 v 已经在覆盖集中）
- 将每个团集 C_j 中除了顶点 u 之外的其余 $n_j - 1$ 个顶点也加入顶点覆盖集中，显然这些顶点能覆盖团集中所有边。
- 显然，顶点覆盖集的大小为 k 。
- 因此，如果 f 是可满足的，则图 G 有一个大小为 k 的顶点覆盖。

可满足性 ∞_{poly} 顶点覆盖

$\leq$:

假设图G有一个大小为k的顶点覆盖集。

- 显然在步骤 (1) 构造的每个边 $(x_i, \sim x_i)$ 中至少有1个顶点必须在覆盖集中。因此可知已有n个顶点在覆盖集中，同时，这些顶点为端点的连通边也由这些顶点所覆盖了。
- 考虑剩余的 $(n_1-1)-(n_2-1)-\dots-(n_m-1)$ 个顶点。不难看出，对于一个大小为 n_j 的团集，顶点覆盖集的大小为 n_j-1 。因此，对每个团集，由于连通边除了与连通边有关的顶点外，其余顶点都包含在顶点覆盖中。
- 对于f中的每个变元 x_i ，如果它对应的顶点在覆盖集中，则令 $x_i=\text{true}$ ，否则，令 $x_i=\text{false}$ 。这样，在每个子句对应的团集中，一定有一个顶点连接 x_i 或 $\sim x_i$ 对应赋值为真的顶点，才能使连通边被覆盖。
- 因此，每个子句中至少有1个文字的赋值为真。
- 即f是可满足的。

- 
- 在上述构件证明法中，设计了两种“构件”：变元构件和简单析取式构件。
 - 变元构件包括一对顶点和连接它们的边；
 - 简单析取式构件为团集。用这些构件以及构件之间的连接组成了图 G 。每个构件各有功能，通过这种方式达到用VC的实例表达SAT实例的目的。



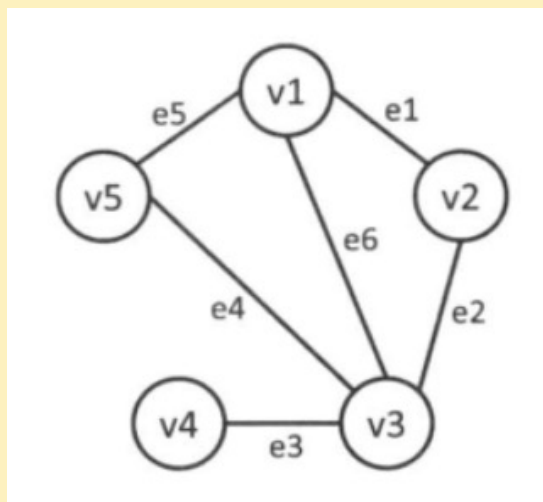
7.5.4 问题等价性

- 问题等价性是指已有定理来保证问题之间可以在多项式时间内相互转换。
- 因此，如果已证明其中某个问题是NP完全的，可以很容易证明与之等价的问题也是NP完全的。

顶点覆盖 $\propto_{\text{poly}}$ 独立集

引理 设 $G=(V,E)$ 是连通无向图，那么 $S \subseteq V$ 是一个独立集当且仅当 $V-S$ 是 G 的一个顶点覆盖。

证明 设 $e=(u,v)$ 是 G 中的任意边， S 是一个独立集当且仅当 u 或 v 至少有一个在 $V-S$ 中，即 $V-S$ 是 G 中的顶点覆盖。



- $| \text{最大独立集} | + | \text{最小点覆盖} | = |V|$
- 最小顶点覆盖为 $\{v1, v3\}$
- 最大独立集 $\{v2, v4, v5\}$